

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. (BL :3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25 Duration : 1 Hour

Constraint Based Problem

Code of Conduct:

*I B.JAYANTH(192511375) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
understand that any violation of academic integrity rules will result in disciplinary action.*

B.JAYANTH

Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Design a Pushdown Automaton that accepts the language

$$L = \{ a^n b^n \mid n \geq 1 \}$$

Constraint: You are allowed to use only one stack symbol (besides initial symbol)

2. CFG to PDA Conversion with Limited Productions

Given a Context-Free Grammar:

$$S \rightarrow aSb \mid ab$$

Construct an equivalent PDA.

Constraint: PDA must be constructed using only 2 states

3. PDA to CFG Conversion with Restricted Variables

Convert a given PDA into an equivalent CFG.

Constraint: The resulting CFG must use at most 3 variables (non-terminals)

4. Turing Machine with Limited Tape Movement

Design a Turing Machine to recognize the language

$$L = \{ ww \mid w \in \{0,1\}^* \}$$

Constraint: The tape head can move only to the right (no left movement allowed)

5. Consider a language $L = \{ a^n b^n c^n \mid n \geq 1 \}$

Constraint: Design a Turing Machine for L

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING ASSESSMENT TOOLS

QUESTION 1

DFA-Based String Validation

Question

Design and implement a C-based DFA validation module that accepts strings which **begin with a and end with a**. The program should accept input dynamically, process it using DFA transitions, and display Accepted or Rejected.

1. Given / Required:

Language:

$L = \{ w \mid w \text{ begins with 'a' and ends with 'a' } \}$

Input alphabet:

$\Sigma = \{a, b\}$

The DFA must:

1. Read a string from the user.
2. Process each character using state transitions.
3. Accept if the string starts with 'a' and ends with 'a'.
4. Otherwise, reject the string.

2. DFA States:

q0 = Initial state

q1 = String starts with 'a' and currently ends with 'a'

q2 = String starts with 'a' and currently ends with 'b'

qdead = String starts with 'b'

Initial state:

q0

Final state:

q1

3. Transition Table

Current State	Input a	Input b
q0	q1	qdead

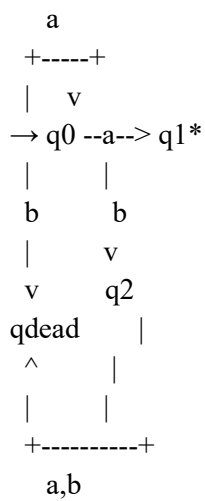
SIMATS ENGINEERING ASSESSMENT TOOLS

Current State	Input a	Input b
q1	q1	q2
q2	q1	q2
qdead	qdead	qdead

Explanation

- From q0, the first character must be a.
- q1 is the accepting state because the current string ends in a.
- Reading b from q1 moves to q2.
- Reading a from q2 returns to q1.
- qdead permanently rejects the string.

4. State Diagram



q1 --a--> q1

q2 --a--> q1

q2 --b--> q2

q0 = Initial state

q1* = Final state

qdead = Dead state

5. Example

Consider:

Input = abba

State transitions:

q0

→ q1 (read a)

→ q2 (read b)

→ q2 (read b)

→ q1 (read a)

Final state:

q1

Therefore:

Accepted

For:

Input = abb

the transitions are:

q0 → q1 → q2 → q2

Since q2 is not a final state:

Rejected

6. C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int state = 0;
```

```
    int i;
```

```
    // Read the input string
```

```
    printf("Enter a string: ");
```

```
    scanf("%s", str);
```

```
    // Process each character
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
for (i = 0; str[i] != '\0'; i++)
{
    if (state == 0)    // q0
    {
        if (str[i] == 'a')
            state = 1;    // q0 -> q1
        else
            state = 3;    // q0 -> qdead
    }
    else if (state == 1) // q1
    {
        if (str[i] == 'a')
            state = 1;    // q1 -> q1
        else
            state = 2;    // q1 -> q2
    }
    else if (state == 2) // q2
    {
        if (str[i] == 'a')
            state = 1;    // q2 -> q1
        else
            state = 2;    // q2 -> q2
    }
    else    // qdead
    {
        state = 3;
    }
}

// q1 is the accepting state
if (state == 1)
    printf("Accepted\n");
else
    printf("Rejected\n");

return 0;
```

SIMATS ENGINEERING ASSESSMENT TOOLS

}

Sample Output

Enter a string: abba

Accepted

Enter a string: abb

Rejected

Result

The DFA and C program successfully accept strings that **begin with a and end with a**, and reject all other strings

QUESTION 2

Design a Pushdown Automaton (PDA)

Question

Design a Pushdown Automaton that accepts the language described in Question 2 of the document, involving the **number of 0s and 1s in w**. The extracted text of the document does not contain the comparison condition completely.

1. Given / Required:

Assuming the intended language is:

$$L = \{ w \in \{0,1\}^* \mid n_0(w) = n_1(w) \}$$

where:

$n_0(w)$ = Number of 0's in w

$n_1(w)$ = Number of 1's in w

The PDA must accept when the total number of 0s is equal to the total number of 1s.

2. PDA Approach:

The stack stores the **unmatched symbols**.

If input = 0 and stack top = 1 $\rightarrow$ Pop 1

SIMATS ENGINEERING ASSESSMENT TOOLS

If input = 1 and stack top = 0 $\rightarrow$ Pop 0

If the same type of symbol is encountered $\rightarrow$ Push it

After the complete input is read:

Only Z0 in stack $\rightarrow$ Accept

3. PDA Definition:

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

where:

$Q = \{q_0, q_f\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, Z_0\}$

q_0 = Initial state

Z_0 = Initial stack symbol

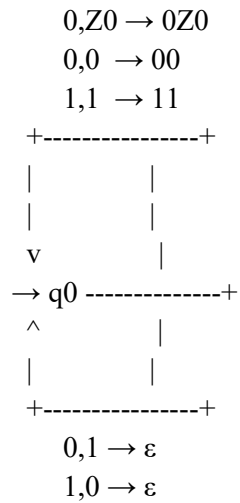
$F = \{q_f\}$

4. Transition Table:

Current State	Input	Stack Top	Next State	Stack Operation
q_0	0	Z_0	q_0	Push 0
q_0	0	0	q_0	Push 0
q_0	0	1	q_0	Pop 1
q_0	1	Z_0	q_0	Push 1
q_0	1	1	q_0	Push 1
q_0	1	0	q_0	Pop 0
q_0	ϵ	Z_0	q_f	Accept

5. State Diagram:

SIMATS ENGINEERING ASSESSMENT TOOLS



$q0 \xrightarrow{\varepsilon} Z0 \rightarrow Z0 \rightarrow ((qf))$

$q0$ = Initial state

qf = Final state

$Z0$ = Bottom stack symbol

6. Working Example:

Consider:

$w = 0011$

Initial configuration:

$(q0, 0011, Z0)$

Read the first 0:

$(q0, 011, 0Z0)$

Read the second 0:

$(q0, 11, 00Z0)$

Read the first 1 and pop 0:

$(q0, 1, 0Z0)$

Read the second 1 and pop 0:

$(q0, \varepsilon, Z0)$

Move to the final state:

$(qf, \varepsilon, Z0)$

SIMATS ENGINEERING ASSESSMENT TOOLS

Therefore:

0011 $\rightarrow$ Accepted

7. Result:

The PDA accepts a string when:

Number of 0's = Number of 1's

For example:

01 $\rightarrow$ Accepted

0011 $\rightarrow$ Accepted

0101 $\rightarrow$ Accepted

001 $\rightarrow$ Rejected

QUESTION 3

Design a Pushdown Automaton (PDA)

Question

Design a Pushdown Automaton (PDA) for the language:

$L = \{ a^n b^{2n} \mid n \geq 1 \}$

The PDA must accept strings in which the number of bs is **twice the number of as**.

1. Given / Required:

$L = \{ a^n b^{2n} \mid n \geq 1 \}$

Therefore:

Number of b's = $2 \times$ Number of a's

Examples:

ab $\rightarrow$ Accepted

aabbbb $\rightarrow$ Accepted

aaabbbbb $\rightarrow$ Accepted

Invalid examples:

a $\rightarrow$ Rejected

abb $\rightarrow$ Rejected

aabbb $\rightarrow$ Rejected

SIMATS ENGINEERING ASSESSMENT TOOLS

2. PDA Approach:

For every a read, the PDA pushes **two X symbols** onto the stack.

Then, for every b read, the PDA pops **one X**.

Therefore:

1 a $\rightarrow$ 2 X

2 a $\rightarrow$ 4 X

3 a $\rightarrow$ 6 X

If all Xs are removed exactly when the input ends, the string is accepted.

3. PDA Definition:

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

where:

$Q = \{q_0, q_1, q_f\}$

$\Sigma = \{a, b\}$

$\Gamma = \{X, Z_0\}$

q_0 = Initial state

Z_0 = Initial stack symbol

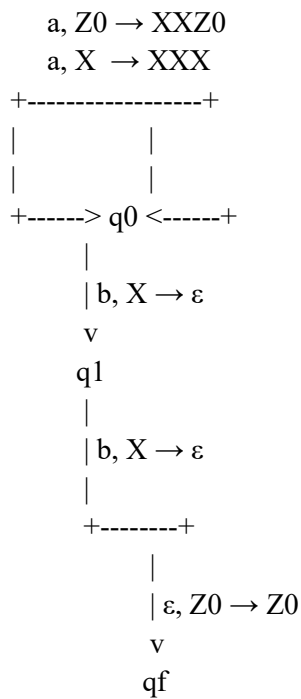
$F = \{q_f\}$

4. Transition Table:

Current State	Input	Stack Top	Next State	Stack Operation
q0	a	Z0	q0	Push XX
q0	a	X	q0	Push XX
q0	b	X	q1	Pop X
q1	b	X	q1	Pop X
q1	ϵ	Z0	qf	Accept

The PDA first processes the as and then processes the bs.

5. State Diagram:



q0 = Initial state

q1 = Processing b's

qf = Final state

Z0 = Bottom stack symbol

6. Step-by-Step Working:

Consider:

w = aabbbb

Here:

Number of a's = 2

Number of b's = 4

Since:

$$4 = 2 \times 2$$

the string should be accepted.

SIMATS ENGINEERING ASSESSMENT TOOLS

Step 1: Read first a

Push two Xs.

Input remaining = abbbb

Stack = XXZ0

Step 2: Read second a

Push two more Xs.

Input remaining = bbbb

Stack = XXXXZ0

Step 3: Read first b

Pop one X.

Input remaining = bbb

Stack = XXXZ0

Step 4: Read second b

Input remaining = bb

Stack = XXZ0

Step 5: Read third b

Input remaining = b

Stack = XZ0

Step 6: Read fourth b

Input remaining = ϵ

Stack = Z0

All four Xs have been removed.

Therefore:

aabbbb $\rightarrow$ Accepted

7. Instantaneous Description:

$(q_0, aabbbb, Z0)$

$\vdash (q_0, abbbb, XXZ0)$

$\vdash (q_0, bbbb, XXXXZ0)$

$\vdash (q_1, bbb, XXXZ0)$

$\vdash (q_1, bb, XXZ0)$

$\vdash (q_1, b, XZ0)$

$\vdash (q_1, \epsilon, Z0)$

$\vdash (q_f, \epsilon, Z0)$

Hence:

SIMATS ENGINEERING ASSESSMENT TOOLS

$aabbbb \in L$

8. Rejected Example:

Consider:

$w = aabbb$

Here:

Number of a's = 2

Number of b's = 3

Required number of bs:

$$2 \times 2 = 4$$

But only three bs are present.

After processing the input:

Stack = XZ0

One X remains.

Therefore:

$aabbb \rightarrow \text{Rejected}$

9. Result:

The PDA accepts exactly those strings of the form:

$a^n b^{2n}$

where:

$$n \geq 1$$

and:

Number of b's = $2 \times$ Number of a's

Thus:

$ab \rightarrow \text{Accepted}$

$aabbbb \rightarrow \text{Accepted}$

$aaabbbbb \rightarrow \text{Accepted}$

QUESTION 4

C-Based ε -Closure Module for an NFA

SIMATS ENGINEERING ASSESSMENT TOOLS

Question

Develop a C-based computational module to determine the **ϵ -closure of all states** in a Non-Deterministic Finite Automaton (NFA) with ϵ -transitions.

1. Given / Required:

The program must:

1. Represent ϵ -transitions of the NFA.
2. Find the ϵ -closure of every state.
3. Include the state itself in its ϵ -closure.
4. Follow all possible ϵ -transitions.
5. Display the ϵ -closure of each state.

Definition

The ϵ -closure of a state is the set of states reachable from that state using **zero or more ϵ -transitions**.

For example:

$q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2$

Then:

$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$

$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$

$\epsilon\text{-closure}(q_2) = \{q_2\}$

2. NFA Representation:

An adjacency matrix is used to represent ϵ -transitions.

$1 \rightarrow \epsilon\text{-transition exists}$

$0 \rightarrow \epsilon\text{-transition does not exist}$

For example:

SIMATS ENGINEERING ASSESSMENT TOOLS

From / To	q0	q1	q2	q3
q0	0	1	0	0
q1	0	0	1	0
q2	0	0	0	1
q3	0	0	0	0

This represents:

q0 $\xrightarrow{\epsilon}$ q1

q1 $\xrightarrow{\epsilon}$ q2

q2 $\xrightarrow{\epsilon}$ q3

3. ϵ -Closure Table

State	ϵ -Reachable States	ϵ -Closure
q0	q1, q2, q3	{q0, q1, q2, q3}
q1	q2, q3	{q1, q2, q3}
q2	q3	{q2, q3}
q3	None	{q3}

The state itself is included because ϵ -closure allows **zero ϵ -transitions**.

4. Algorithm:

Step 1: Read the number of states.

Step 2: Read the ϵ -transition matrix.

Step 3: For each state, add the state itself to its ϵ -closure.

Step 4: Follow every possible ϵ -transition.

Step 5: Add each newly reached state to the closure.

SIMATS ENGINEERING ASSESSMENT TOOLS

Step 6: Continue until no new state can be reached.

Step 7: Display the ε -closure of every state.

5. C Program:

```
#include <stdio.h>

#define MAX 20

int epsilon[MAX][MAX];
int closure[MAX][MAX];
int n;

/* Find epsilon-closure of a state */
void findEpsilonClosure(int state)
{
    int stack[MAX];
    int top = -1;
    int current;
    int i;

    /* Include the state itself */
    closure[state][state] = 1;
    stack[++top] = state;

    /* Follow all epsilon transitions */
    while (top != -1)
    {
        current = stack[top--];

        for (i = 0; i < n; i++)
        {
            if (epsilon[current][i] == 1 &&
                closure[state][i] == 0)
            {
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
        closure[state][i] = 1;
        stack[++top] = i;
    }
}
}

int main()
{
    int i, j;

    printf("Enter number of states: ");
    scanf("%d", &n);

    printf("Enter epsilon transition matrix:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &epsilon[i][j]);
        }
    }

    /* Calculate epsilon-closure for every state */
    for (i = 0; i < n; i++)
    {
        findEpsilonClosure(i);
    }

    /* Display the results */
    printf("\nEpsilon-Closure of all states:\n");

    for (i = 0; i < n; i++)
    {
        printf("E-Closure(q%d) = { ", i);
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
for (j = 0; j < n; j++)  
{  
    if (closure[i][j] == 1)  
    {  
        printf("q%d ", j);  
    }  
}  
  
printf("{}\n");  
}  
  
return 0;  
}
```

6. Sample Input:

Enter number of states: 4

Enter epsilon transition matrix:

0 1 0 0

0 0 1 0

0 0 0 1

0 0 0 0

7. Sample Output:

Epsilon-Closure of all states:

E-Closure(q0) = { q0 q1 q2 q3 }

E-Closure(q1) = { q1 q2 q3 }

E-Closure(q2) = { q2 q3 }

E-Closure(q3) = { q3 }

8. Result:

SIMATS ENGINEERING ASSESSMENT TOOLS

The C program successfully determines and displays the **ϵ -closure of every state** by following all possible ϵ -transitions in the NFA.

QUESTION 5

C-Based CFG Validation Module

Question

Develop a C-based input validation module for a binary string based on the following Context-Free Grammar (CFG):

$S \rightarrow 0A1$

$A \rightarrow 0A \mid 1A \mid \epsilon$

The module should determine whether the given binary string conforms to the grammar.

1. Given / Required:

Grammar

$S \rightarrow 0A1$

$A \rightarrow 0A \mid 1A \mid \epsilon$

The required program should:

1. Read a binary string.
2. Check whether it contains only 0 and 1.
3. Validate the string according to the given CFG.
4. Display Accepted or Rejected.

2. Grammar Interpretation:

From:

$S \rightarrow 0A1$

the string must begin with:

0

and end with:

1

The productions:

$A \rightarrow 0A$

$A \rightarrow 1A$

allow 0 or 1 to occur in the middle.

SIMATS ENGINEERING ASSESSMENT TOOLS

The production:

$A \rightarrow \epsilon$

terminates the derivation.

Therefore, the strings generated by the grammar have the form:

0^w1

where w consists of zero or more 0s and 1s.

3. Validation Conditions:

Condition	Requirement
Input symbols	Only 0 and 1
First symbol	0
Last symbol	1
Middle portion	Any combination of 0 and 1
Empty string	Rejected

4. Algorithm:

Step 1: Read the binary string.

Step 2: Check whether the string is empty.

Step 3: Check every character.

If any character is not 0 or 1, reject.

Step 4: Check whether the first character is 0.

Step 5: Check whether the last character is 1.

Step 6: If all conditions are satisfied, accept the string.

Step 7: Otherwise, reject the string.

5. C Program:

SIMATS ENGINEERING ASSESSMENT TOOLS

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100];
    int i;
    int valid = 1;
    int length;

    // Read the input string
    printf("Enter a binary string: ");
    scanf("%s", str);

    length = strlen(str);

    // Check for valid binary symbols
    for (i = 0; i < length; i++)
    {
        if (str[i] != '0' && str[i] != '1')
        {
            valid = 0;
            break;
        }
    }

    // Check first symbol
    if (valid && str[0] != '0')
    {
        valid = 0;
    }

    // Check last symbol
    if (valid && str[length - 1] != '1')
    {
        valid = 0;
    }
}
```

SIMATS ENGINEERING ASSESSMENT TOOLS

```
}  
  
// Display result  
if (valid)  
    printf("Accepted\n");  
else  
    printf("Rejected\n");  
  
return 0;  
}
```

6. Derivation Example:

Consider:

Input = 001

Derivation:

S

→ 0A1

→ 00A1

→ 00ε1

→ 001

Therefore:

001 → Accepted

7. Examples:

Input	First Symbol	Last Symbol	Result
01	0	1	Accepted
001	0	1	Accepted
011	0	1	Accepted
0101	0	1	Accepted
10	1	0	Rejected
00	0	0	Rejected
11	1	1	Rejected

SIMATS ENGINEERING ASSESSMENT TOOLS

8. Sample Execution:

Input

Enter a binary string: 0101

Output

Accepted

For:

Enter a binary string: 1010

Output:

Rejected

Result:

The C-based validation module checks a binary string according to the given CFG:

$S \rightarrow 0A1$

$A \rightarrow 0A \mid 1A \mid \varepsilon$

and accepts strings generated by the grammar while rejecting invalid strings.